

Práctica 1

Generación de datos sintéticos y almacenamiento de información

Este documento responde al enunciado de la práctica, explicando cómo se ha implementado cada requisito y aportando conclusiones sobre los distintos formatos y SGBD para comprender sus ventajas y limitaciones en situaciones reales.

El código fuente se ha organizado de manera modular para separar responsabilidades y facilitar el mantenimiento. La estructura del proyecto es la siguiente:

```
BDIA2425-lab1-EvaBlazquez_GabrielaDamas/
├─ data/ → Archivos de referencia CSV
├─ src/ → Código fuente principal
│   ├── main.py → Control de ejecución y orquestación
│   ├── generators.py → Generación de usuarios y vehículos
│   ├── providers.py → Proveedores personalizados
│   ├── io_utils.py → Lectura/escritura en distintos formatos
│   └── db_utils.py → Conexión y operaciones con bases de datos
└─ output/ → Ficheros generados
```

Esta organización facilita la reutilización y claridad del código, permitiendo modificar o ampliar componentes de forma independiente.

1. Generación de datos sintéticos

Para generar datos sintéticos realistas se empleó la librería *Faker*, complementada con *providers* personalizados y archivos CSV auxiliares. Esta estrategia permite mantener los datos de referencia separados de la lógica de generación, facilitando su mantenimiento y actualización sin modificar el código.

1.1. Conjuntos de datos y relación

Se generaron dos conjuntos de datos relacionales:

1.1.1. Users (usuarios): Name, DNI (único), Email, PhoneMobile, PhoneLandline, Address, City, PostalCode, Province.

1.1.2. Vehicles (vehículos): Plate, VIN, Year, Make, Model, Category, UserDNI (clave foránea).

La relación es de tipo 1:N, donde un usuario puede poseer entre 0 y 3 vehículos, y cada vehículo pertenece a un único usuario identificado mediante UserDNI.

1.2. Estrategia:

Antes de crear usuarios y vehículos, el programa inicializa el generador principal `build_generators()`, que fija una semilla, construye la instancia de Faker y registra los providers propios (DNI, matrícula y VIN); así, `generate_users()` y `generate_vehicles()` se centran en la lógica de creación de entidades.

1.2.1. Users:

Para la generación de usuarios, el DNI se obtiene mediante un *DNIPProvider*, que genera un número aleatorio de ocho dígitos y calcula su letra de control según el algoritmo oficial.

La dirección se genera con `fake.street_address()`, mientras que el código postal se elige aleatoriamente de `codigos_postales_municipios.csv` y de ahí se asocia el municipio correspondiente. A partir de los primeros dos dígitos del código postal, se obtiene la provincia en `prov_tlf.csv`, que también proporciona el prefijo telefónico provincial.

En cuanto a los teléfonos, el móvil se genera aleatoriamente con nueve dígitos, pero siempre comienza por 6 o 7, mientras que el teléfono fijo utiliza el prefijo provincial, con una probabilidad aproximada del 53,9 % de estar presente, reflejando la proporción real de hogares con línea fija.

Por último, el correo electrónico se construye, para mayor coherencia, a partir del nombre y apellido del usuario, pero el dominio se obtiene con `fake.free_email_domain()`, y se garantiza su unicidad añadiendo un número incremental si ya existe uno igual.

1.2.2. Vehicles:

Cada vehículo se vincula a un usuario mediante su DNI, asegurando que cada registro tenga un propietario único. Asimismo, cada usuario recibe entre 0 y 3 vehículos según una probabilidad predefinida, simulando la distribución real de propiedad de automóviles (es más frecuente poseer un solo coche que varios).

De manera similar, en la generación de vehículos, para generar una matrícula válida en formato español se utiliza *PlateProvider*, que se apoya en el archivo `series_matriculas.csv`, que recoge las series de letras asignadas a cada año desde 2000 hasta 2025. Por otro lado, el archivo `models_by_make.csv` define las marcas, modelos y categorías de los coches, asegurando que la combinación marca-modelo sea coherente y realista.

Para el número de bastidor se utiliza *VINProvider*, que genera identificadores únicos de 17 caracteres siguiendo el estándar internacional, combinando el código de fabricante (WMI), el año de fabricación y un dígito de control calculado mediante un sistema de pesos y transliteración de caracteres. Además, los años de fabricación se asignan siguiendo una distribución ponderada que otorga mayor peso a los años intermedios, evitando que los coches sean muy recientes o antiguos.

La generación se orquesta desde el archivo `main.py`, donde se definen el tamaño del dataset (`--n_users`) y la semilla (`--seed`) para reproducibilidad, además de la exportación a formatos y la carga en BBDD.

2. Almacenamiento de la información generada en ficheros

El proyecto genera los conjuntos de datos de users y vehicles en diferentes formatos:

En los formatos **CSV** y **Parquet**, se crea un fichero por cada conjunto, conectados mediante el DNI (en vehículos, el campo `UserDNI`). El formato **CSV** se escribe de forma secuencial con la librería estándar de Python y una barra de progreso; es el formato más simple y compatible, ideal para revisar datos rápidamente o abrirlos en programas como Excel.

En cambio, **Parquet** se genera con la librería *PyArrow* y utiliza compresión para reducir el tamaño del fichero. Al estar orientado por columnas, permite leer solo las partes necesarias y es mucho más eficiente cuando se trabaja con grandes volúmenes de datos o se hacen análisis estadísticos.

Para **JSON**, se implementaron dos versiones. En la versión anidada (`users_nested.json`), cada usuario contiene dentro de su registro un listado con sus vehículos, lo que facilita ver toda la información de una persona en un solo bloque. En la versión separada (`users.json` y `vehicles.json`), los datos se guardan en ficheros distintos, pero conectados por el DNI, lo que permite trabajar con usuarios o vehículos por separado. Este método además escribe los datos por partes (sin cargar todo en memoria), lo que mejora el rendimiento si los conjuntos son grandes.

El formato **Avro** repite ambos enfoques (anidado y separado), pero añade un elemento importante: el uso de esquemas. Estos esquemas definen el tipo y la estructura de los datos, de manera que cualquier programa que lea los ficheros pueda validar si la información es correcta. Además, Avro comprime los datos y los guarda en binario, logrando un equilibrio entre tamaño, rapidez y compatibilidad. Es especialmente útil cuando los datos se van a usar o transmitir entre sistemas distintos.

2.1. Características más destacadas de cada formato de fichero.

2.1.1. CSV: Es un formato de texto sencillo y muy compatible. Se puede abrir fácilmente con cualquier editor o programa de hojas de cálculo. Sin embargo, no guarda información sobre los tipos de datos (todo se trata como texto) y puede ocupar más espacio. Es recomendable para conjuntos de tamaño pequeño o mediano y cuando la prioridad es la legibilidad o la compatibilidad.

2.1.2. JSON: Es también texto legible, pero más flexible porque permite guardar estructuras anidadas (por ejemplo, un usuario con sus vehículos dentro). Es ideal para intercambiar datos entre programas o aplicaciones web, aunque sus ficheros pueden ser grandes y lentos de procesar en volúmenes altos.

2.1.3. Avro: Es binario y necesita un esquema que describe la forma y el tipo de los datos. Esto garantiza que la información sea válida y consistente. Es más compacto que JSON y adecuado para guardar o enviar grandes volúmenes de información con una estructura definida.

2.1.4. Parquet: Es un formato binario que guarda los datos organizados por columnas, lo que permite comprimir mejor y acceder solo a los campos que se necesitan. Es muy útil para análisis de datos o cuando se trabaja con tablas grandes. En cambio, no es el más cómodo para revisar manualmente o hacer modificaciones pequeñas.

2.2. ¿Usarías cualquier formato de forma indistinta en cualquier situación? ¿En qué situaciones resultan más aconsejables cada uno de ellos?

Cada formato tiene su situación ideal:

2.2.1. CSV: cuando se busca simplicidad, compatibilidad o revisión rápida.

2.2.2. JSON anidado: cuando interesa ver toda la información de un usuario en un único bloque.

2.2.3. JSON separado: cuando se quiere procesar usuarios y vehículos por separado.

2.2.4. Avro: cuando se necesita validar los datos o mantener una estructura fija en el tiempo.

2.2.5. Parquet: cuando se analizan grandes volúmenes de datos o se necesita un formato más eficiente.

2.3. ¿Qué diferencias encuentras entre usar formatos de almacenamiento basados en filas frente a los basados en columnas?

Los formatos por filas (CSV, JSON o Avro) guardan la información completa de cada registro uno detrás de otro. Esto es más rápido cuando se insertan o leen registros completos (por ejemplo, un usuario entero).

Los formatos por columnas (Parquet) guardan los datos agrupados por campo. Esto permite leer solo las columnas necesarias (por ejemplo, solo “Provincia” o “Año del vehículo”) y comprimir mejor, por lo que es más eficiente en análisis de datos grandes. Sin embargo, escribir o modificar datos individuales puede ser más lento.

2.4. ¿Qué ventajas e inconvenientes encuentras entre almacenar la información mediante ficheros separados o en estructura jerárquica?

Guardar la información en ficheros separados (usuarios y vehículos por separado) se parece a una base de datos relacional: evita duplicaciones y facilita trabajar con cada conjunto de datos por su cuenta, pero requiere combinar los ficheros cuando se quiere ver toda la información junta.

En cambio, guardar los datos en una estructura jerárquica o anidada (todo el usuario con sus vehículos dentro) es más cómodo para visualizar la “ficha completa” de una persona, aunque los ficheros pueden ser más grandes y menos prácticos si solo se necesita actualizar o analizar una parte.

En este proyecto, la estructura anidada facilita la lectura completa de los datos de cada usuario, mientras que la estructura separada es más flexible para análisis y procesos de carga en bases de datos.

3. Almacenar la información generada en SGBD

Además de guardar los datos en ficheros, el proyecto permite almacenarlos en diferentes sistemas de bases de datos:

3.1. SQLite

SQLite se utiliza como base de datos local y ligera, ideal para desarrollo o pruebas, ya que guarda toda la información en un único fichero sin requerir configuración de servidor. La función `write_sqlite()` realiza todo el proceso:

3.1.1. Creación de tablas: se definen las tablas `users` y `vehicles` desde cero, con la relación `UserDNI = DNI`.

3.1.2. Eliminación previa: antes de crear las tablas se ejecutan `DROP TABLE IF EXISTS users` y `DROP TABLE IF EXISTS vehicles` para asegurarse de empezar desde una base limpia.

3.1.3. Inserción: los datos se insertan en lotes de tamaño configurable (`batch_size`) y se muestra el progreso con `tqdm`.

De este modo, cada ejecución genera una base de datos nueva y consistente.

3.2. PostgreSQL

Para PostgreSQL, el código está dividido en funciones que dividen cada parte del proceso:

3.2.1. Conexión y creación de la base de datos: `get_connection()` centraliza la conexión y `ensure_database_exists()` comprueba si la base de datos indicada existe; si no, la crea automáticamente conectándose a la base administrativa por defecto (`postgres`). Esta operación utiliza `autocommit=True` porque `CREATE DATABASE` no puede ejecutarse dentro de una transacción.

- 3.2.2. Creación de tablas:** `create_tables()` genera las tablas `users` y `vehicles` si no existen. La tabla de vehículos incluye la clave foránea `user_dni` que garantiza la relación con `users(dni)` y la integridad referencial.
- 3.2.3. Eliminación y limpieza:** `truncate_postgres_tables()` permite vaciar las tablas mediante la instrucción `TRUNCATE TABLE vehicles, users;`, respetando el orden correcto para no violar las claves foráneas. Esto permite repetir ejecuciones sin duplicar información.
- 3.2.4. Inserción:** `insert_into_postgres()` inserta primero los usuarios y después los vehículos, asegurando que las relaciones sean válidas. Utiliza `ON CONFLICT DO NOTHING` para evitar errores por duplicados y `tqdm` para mostrar el progreso durante la carga.

Este flujo garantiza que la base de datos se mantenga limpia, coherente y lista para reutilizarse en ejecuciones posteriores.

3.3. MongoDB

En MongoDB, los datos se almacenan en colecciones separadas `users` y `vehicles`.

- 3.3.1. Conexión:** `get_mongo_client()` gestiona la conexión al servidor, creando el cliente a partir de la URI indicada.
- 3.3.2. Limpieza:** antes de insertar, las colecciones se vacían mediante `delete_many({})`, eliminando cualquier dato previo y evitar acumulaciones y duplicados.
- 3.3.3. Inserción:** `insert_into_mongodb()` realiza `insert_many` en bloques (`batch_size` configurable) para mejorar el rendimiento y evitar sobrecargar la memoria; la base y las colecciones se crean implícitamente si no existen.
- 3.3.4. Relación:** cada vehículo mantiene el campo `UserDNI`, lo que permite vincularlo con su propietario.
- 3.3.5. Cierre:** una vez completada la inserción, se cierra la conexión con `client.close()` para liberar recursos.

Este enfoque es eficiente para manejar grandes volúmenes de información y aprovecha la naturaleza documental de MongoDB, donde las estructuras anidadas son comunes.

Para un modelo 1:N como el nuestro, utilizaríamos PostgreSQL como repositorio principal, SQLite como apoyo ligero para pruebas y distribución; y MongoDB cuando el consumo sea documental, con cambios frecuentes de esquema o necesidades de lectura “por entidad completa”. La doble estrategia de ficheros que usamos (separados vs. anidados) refleja este equilibrio: la versión separada se alinea con lo relacional y la anidada con lo documental, permitiendo elegir la mejor base para cada caso.